

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Data Interpretation

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 1 – Data Interpretation
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Madhavan S	Reg. No.:	192425113
Section / Batch:	CSE AI 2024	Date:	18/08/2026

Code of Conduct

I Madhavan S/192425113 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Madhavan S

Instructions

- Read each data set, table, semantic representation, or scenario carefully before answering.
- Analyze the given information and provide logical, evidence-based interpretations.
- Show all intermediate reasoning, semantic analysis steps, predicate representations, or calculations wherever applicable.
- Answers should be concise, structured, and technically accurate.
- Support your conclusions using the data provided in the question.
- Draw diagrams, semantic networks, parse trees, or logical representations wherever relevant.
- Avoid copying definitions alone; focus on analyzing the given scenario and interpreting the data.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic representations, identify action–entity relationships, detect interpretation errors, and evaluate meaning representation in chatbot systems.	Q1
First-Order Predicate Calculus & Representing Linguistically Relevant Concepts	Construct predicate representations, apply logical inference rules, derive conclusions, and evaluate reasoning in smart manufacturing environments.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze contextual clues, determine correct word senses, evaluate ambiguity resolution, and improve semantic understanding in search systems.	Q3
Syntax-Driven Semantic Analysis	Analyze syntactic structures, assign semantic roles, evaluate parsing accuracy, and improve semantic interpretation in healthcare NLP applications.	Q4

Annexure A – Data Interpretation

1. Semantic Representation in Customer Support Chatbots.

A telecom company collected customer chatbot interactions. The semantic analyzer generated the following meaning representations.

Customer Query	Semantic Representation
"Activate international roaming for my number."	ACTIVATE(Roaming, Customer)
"Deactivate caller tune service."	DEACTIVATE(CallerTune, Customer)
"Check my data balance."	QUERY(DataBalance, Customer)
"Enable 5G service."	ACTIVATE(5GService, Customer)

Additional chatbot logs show:

Query ID	Actual User Intent	Predicted Intent
Q1	Activate Roaming	Activate Roaming
Q2	Deactivate Caller Tune	Activate Caller Tune
Q3	Check Data Balance	Query Data Balance
Q4	Enable 5G Service	Activate 5G Service

Tasks:

1. Analyze the semantic representations and identify the action-object relationships.
2. Determine which query contains semantic interpretation errors.
3. Evaluate how meaning representation affects chatbot decision accuracy.
4. Recommend improvements to enhance semantic understanding in telecom customer support systems.

Answer:

1. Action-object relationships: each meaning representation follows a two-argument predicate frame ACTION(Object, Agent), where the action names the requested state change, the first argument is the Theme/service affected, and the second argument is the Agent making the request.

- “Activate international roaming for my number” → ACTIVATE(Roaming, Customer) — action=ACTIVATE, object=Roaming, agent=Customer.
- “Deactivate caller tune service” → DEACTIVATE(CallerTune, Customer) — action=DEACTIVATE, object=CallerTune, agent=Customer.
- “Check my data balance” → QUERY(DataBalance, Customer) — action=QUERY (an information-retrieval act rather than a state change), object=DataBalance, agent=Customer.
- “Enable 5G service” → ACTIVATE(5GService, Customer) — action=ACTIVATE, object=5GService, agent=Customer.
- All four frames share the same two-slot structure, which shows the analyzer is consistently extracting a single service entity and a single action label per utterance, but is not yet capturing polarity/negation robustly, as seen below.

2. Semantic interpretation errors: comparing Actual Intent to Predicted Intent for each logged query shows only one true semantic error.

- Q1 – Activate Roaming vs Activate Roaming: match, no error.
- Q2 – Deactivate Caller Tune vs Activate Caller Tune: mismatch — the predicted action is the direct opposite (antonym) of the actual action. This is a genuine semantic interpretation error because it reverses the polarity of the request.

- Q3 – Check Data Balance vs Query Data Balance: surface mismatch only — “Check” and “Query” are synonymous information-retrieval verbs referring to the same underlying act, so the meaning is preserved. Not a semantic error, only a lexical/labeling variation.
- Q4 – Enable 5G Service vs Activate 5G Service: same reasoning as Q3 — “Enable” and “Activate” are synonyms for the same action, so meaning is preserved.
- Conclusion: Q2 is the only query with a true semantic interpretation error; Q3 and Q4 are false positives if flagged by a naive string-matching evaluation.

3. Impact of meaning representation on chatbot decision accuracy.

- The chatbot executes actions directly from the predicted predicate, so the semantic layer is the single point of failure between what the customer said and what the system does.
- In Q2, executing ACTIVATE instead of DEACTIVATE turns a service ON that the customer explicitly wanted turned OFF, causing an unwanted subscription/billing charge, a poor customer experience, and a likely repeat contact or complaint.
- In Q3/Q4, because the underlying action is preserved despite different surface verbs, decision accuracy is unaffected — this distinguishes “lexical accuracy” (exact string match) from “actionable/semantic accuracy” (does the executed action match customer intent); the latter is what actually matters for chatbot decision quality.
- This demonstrates that evaluation metrics for chatbot NLU must be built on canonicalized/normalized intents, not raw predicted labels, or true errors like Q2 can be masked by false-positive noise from Q3/Q4-style synonym mismatches.

4. Recommendations to enhance semantic understanding in telecom customer support systems.

- Data level: augment training data with antonym/negation-contrast pairs (activate vs. deactivate, enable vs. disable, subscribe vs. unsubscribe) so the model learns to separate action polarity from action topic.
- Model level: add a dedicated negation/polarity-detection sub-module or classifier head that runs alongside intent classification, rather than relying on a single end-to-end intent label.
- Normalization layer: maintain a canonical intent ontology that maps synonymous verbs (Check=Query, Enable=Activate) to one canonical action, so evaluation and downstream logic are not confused by lexical variation.
- System/UX level: require an explicit confirmation step before executing any irreversible or billing-relevant state change (e.g., “You want to deactivate Caller Tune — confirm?”), which catches residual misclassifications before they take effect.
- Process level: log every predicted-vs-actual mismatch from customer corrections or complaints, and feed it back into periodic retraining (active learning loop) so systematic polarity errors like Q2 are progressively eliminated.

2. First-Order Predicate Calculus for Smart Manufacturing.

An Industry 4.0 manufacturing plant monitors machines using semantic rules.

Production Data:

Machine	Status
M1	Active
M2	Active
M3	Maintenance
M4	Active

Predicate Rules:

- $\text{Active}(x) \rightarrow \text{Producing}(x)$
- $\text{Produces}(x,y) \wedge \text{Active}(x) \rightarrow \text{Available}(y)$
- $\text{Maintenance}(x) \rightarrow \neg \text{Producing}(x)$

Tasks:

1. Represent the production data using First-Order Predicate Calculus.
2. Analyze which products are currently available.
3. Infer whether Gear production is affected by maintenance activities.
4. Evaluate the effectiveness of predicate logic in industrial decision-support systems.

Answer:

1. Representing the production data in First-Order Predicate Calculus.

- Base facts extracted directly from the status table: Active(M1), Active(M2), Maintenance(M3), Active(M4).
- Applying Rule 1, $\forall x [\text{Active}(x) \rightarrow \text{Producing}(x)]$, by modus ponens gives: Producing(M1), Producing(M2), Producing(M4).
- Applying Rule 3, $\forall x [\text{Maintenance}(x) \rightarrow \neg \text{Producing}(x)]$, gives: $\neg \text{Producing}(M3)$.
- Combined knowledge base: $\text{Active}(M1) \wedge \text{Active}(M2) \wedge \text{Maintenance}(M3) \wedge \text{Active}(M4) \wedge \text{Producing}(M1) \wedge \text{Producing}(M2) \wedge \text{Producing}(M4) \wedge \neg \text{Producing}(M3)$.

2. Analyzing which products are currently available.

- Rule 2, $\forall x \forall y [\text{Produces}(x,y) \wedge \text{Active}(x) \rightarrow \text{Available}(y)]$, needs a Produces(machine, product) fact for each machine before Available(y) can be derived — this mapping is not given explicitly in the data set, so it must be assumed or supplied externally.
- Given such a mapping, for every machine that is Active (M1, M2, M4), any product y it Produces becomes Available(y) by modus ponens: e.g., if Produces(M1,P1), then Available(P1); similarly for M2 and M4's products.
- For M3, because $\neg \text{Producing}(M3)$ holds, Rule 2 cannot fire for any product uniquely tied to M3, so under the closed-world assumption those products are NOT available — unless the same product is also produced by another currently active machine, in which case Available(y) still holds via that alternate source.
- This highlights a key practical point: an FOPC monitoring system is only as good as its Produces(x,y) knowledge base; missing machine-to-product mappings silently block correct availability inference.

3. Inferring whether Gear production is affected by maintenance activities.

- The data set does not state explicitly which machine manufactures Gears, so the inference must proceed by cases (a form of proof by cases over the possible Produces(x, Gear) facts).
- Case (a) — M3 is the only machine with Produces(M3, Gear): then $\neg \text{Producing}(M3)$ blocks Rule 2 from firing, so Available(Gear) cannot be derived, and Gear production IS directly affected/halted by the maintenance activity.
- Case (b) — Gear is also produced by an active machine, e.g. Produces(M1, Gear) or Produces(M4, Gear): then Available(Gear) still holds via that active machine's instance of Rule 2, so overall Gear supply continues, though total throughput/capacity is reduced by M3's downtime.
- Without an explicit Produces(x, Gear) fact, a sound reasoner should not assert Available(Gear) or $\neg \text{Available}(Gear)$ outright — it should report the conditional result above, which is itself the correct and honest output of the inference.

4. Evaluating the effectiveness of predicate logic in industrial decision-support systems.

- Strengths: predicate logic gives fully transparent, explainable, and auditable decisions — every conclusion (e.g., $\neg \text{Producing}(M3)$) can be traced back through an explicit rule and fact, which is valuable for regulatory compliance, root-cause analysis, and handing off to non-technical stakeholders.
- Strengths: it is deterministic and consistent — the same facts always yield the same conclusions, and domain experts can add or edit rules directly without retraining a statistical model.
- Limitations: it is brittle to incomplete knowledge, as shown above with the missing Gear $\rightarrow$ machine mapping — the system cannot answer confidently when a needed fact (like Produces(x, Gear)) is simply absent.

- Limitations: it cannot natively represent degrees of certainty or partial states (e.g., a machine running at 80% capacity, or a probabilistic sensor reading), and rules must be manually maintained as the plant's machines and products change over time.
- Limitations: it scales poorly to high-volume, noisy, real-time sensor streams typical of Industry 4.0 environments, where sensor noise and rapid state changes overwhelm a purely symbolic rule engine.
- Recommendation: use predicate logic for the explainable, compliance-facing decision layer, but pair it with probabilistic or machine-learning-based anomaly detection and a well-maintained knowledge graph of machine-product relationships to close the gaps above.

3. Word Sense Disambiguation in E-Commerce Search Engines.

An e-commerce company observed the following search logs.

User Query	Possible Sense 1	Possible Sense 2
"Apple accessories"	Fruit	Technology Brand
"Mouse wireless"	Animal	Computer Device
"Java tutorial"	Island	Programming Language
"Python course"	Snake	Programming Language

Search Context Data:

Query	Clicked Result
Apple accessories	iPhone Charger
Mouse wireless	Bluetooth Mouse
Java tutorial	Coding Lessons
Python course	Software Development Training

Tasks:

1. Analyze the contextual information and determine the correct word sense for each query.
2. Identify the semantic cues used for disambiguation.
3. Evaluate the impact of incorrect sense selection on search engine performance.
4. Suggest an industrial-scale WSD strategy for improving recommendation accuracy.

Answer:

1. Determining the correct word sense for each query using contextual information.

- “Apple accessories” → Technology Brand sense, confirmed by the clicked result “iPhone Charger”; this is a case of homonymy (Apple the fruit vs. Apple the brand share a form but are semantically unrelated).
- “Mouse wireless” → Computer Device sense, confirmed by the clicked result “Bluetooth Mouse”; here “wireless” in the query itself is already a strong technology-domain cue even before the click is observed.
- “Java tutorial” → Programming Language sense, confirmed by “Coding Lessons”; “tutorial” strongly biases toward an instructional/technical sense over the geographic “Island” sense.
- “Python course” → Programming Language sense, confirmed by “Software Development Training”; “course” functions the same way “tutorial” does for Java, overriding the “Snake” sense.
- Method: this is effectively a simplified Lesk-style disambiguation — the sense whose associated vocabulary overlaps most with the query's context words (and, once available, the clicked-result vocabulary) is selected as correct.

2. Identifying the semantic cues used for disambiguation.

- Local lexical context: words immediately co-occurring with the ambiguous term in the query (“wireless,” “tutorial,” “course,” “accessories”) each carry a strong domain bias toward technology.
- Collocation/co-occurrence statistics: in a large query log, “Apple” co-occurs far more often with technology terms (charger, iPhone, MacBook) than with fruit terms in this domain, which can be captured with pointwise mutual information (PMI) or embedding similarity.

- Extralinguistic behavioral signal: the clicked result after the search (iPhone Charger, Bluetooth Mouse, etc.) is direct evidence of what the user actually meant, independent of the text of the query itself.
- Domain/category prior: the fact that the platform is an e-commerce electronics-heavy marketplace raises the prior probability of the technology sense for ambiguous brand/animal/object names even before any context is read.

3. Evaluating the impact of incorrect sense selection on search engine performance.

- Relevance degradation: selecting the wrong sense (e.g., fruit products for “Apple accessories,” pet supplies for “Mouse wireless”) returns results the user never wanted, directly harming perceived search quality.
- Business KPIs: this lowers click-through rate (CTR) and conversion rate on the search results page, increases bounce rate, and can waste ad spend if sponsored listings are shown against the wrong sense.
- User trust and retention: repeated irrelevant results train users to distrust the platform's search bar and switch to manual category browsing or a competitor, which is a long-term churn risk beyond the immediate lost sale.
- Downstream personalization: incorrect sense resolution also pollutes the user's behavioral profile (e.g., logging them as interested in “fruit”), which then degrades future recommendations as well.

4. Suggesting an industrial-scale WSD strategy for improving recommendation accuracy.

- Build a domain-specific sense inventory for the marketplace's known ambiguous terms (Apple, Mouse, Java, Python, and others as they are discovered), rather than relying on a generic dictionary sense list.
- Use contextual embedding models (transformer-based encoders such as BERT-family sentence/query encoders) instead of static keyword or dictionary lookups, since embeddings capture the surrounding query context automatically.
- Treat clickstream/behavioral data as a weak-supervision signal: automatically label historical queries with the sense implied by what users clicked, and use this at scale to fine-tune the disambiguation model without manual annotation.
- Combine the ML model with a lightweight rule/collocation fallback for low-traffic or newly emerging ambiguous terms that don't yet have enough behavioral data.
- Deploy with continuous online evaluation: A/B test sense-routing changes, monitor CTR/conversion by sense, and set up a feedback loop that retrain the model periodically as vocabulary and product catalog evolve (sense drift monitoring).

4. Syntax-Driven Semantic Analysis in Healthcare Information Systems.

A healthcare NLP system processes medical reports.

Parsed Sentences:

Sentence	Parse Structure
Doctor prescribed medicine to patient.	Subject-Verb-Object
Patient reported severe headache.	Subject-Verb-Object
Nurse monitored patient continuously.	Subject-Verb-Object
Medicine reduced blood pressure.	Subject-Verb-Object

Semantic Roles Generated:

Entity	Role
Doctor	Agent
Medicine	Instrument

Patient	Recipient
Headache	Symptom

Tasks:

1. Analyze how syntactic structures contribute to semantic role assignment.
2. Determine whether the assigned semantic roles are appropriate.
3. Evaluate potential errors that could arise from incorrect parsing.
4. Recommend methods to improve syntax-driven semantic analysis in healthcare applications.

Answer:

1. Analyzing how syntactic structures contribute to semantic role assignment.
 - In a Subject-Verb-Object (SVO) parse, syntactic position is the primary cue the system uses to assign a semantic (thematic) role — the Subject NP is mapped to the Agent (the entity performing the action), the main Verb supplies the predicate/action, and the Object NP(s) are mapped to roles such as Theme, Recipient, or Instrument depending on the specific verb's argument structure.
 - Worked example — “Doctor prescribed medicine to patient”: the parse tree gives Subject NP = “Doctor”, VP headed by “prescribed”, direct object NP = “medicine”, and a prepositional phrase “to patient” attached to the VP.
 - Role mapping from that structure: Doctor (Subject) → Agent; medicine (direct Object) → Theme (the entity being transferred/administered); patient (object of “to”) → Recipient/Goal.
 - This shows that semantic role labeling is fundamentally dependent on correct syntactic parsing first — if the parser mis-identifies the subject, object, or PP attachment, the role labels built on top of that structure will also be wrong, regardless of how good the role-labeling model itself is.
2. Determining whether the assigned semantic roles are appropriate.
 - Doctor → Agent: appropriate — the doctor is the volitional entity performing the prescribing action.
 - Patient → Recipient: appropriate — the patient is the entity that receives/benefits from the action.
 - Medicine → Instrument: questionable. An Instrument is typically the means BY WHICH an agent performs an action (e.g., “the doctor treated the wound with a bandage” — bandage = Instrument). Here, medicine is the entity being transferred/administered, which is more accurately a Theme (or “Patient-of-transfer” in some frameworks), not the tool used to perform the prescribing act.
 - Headache → Symptom: this is a useful domain-specific/clinical label but is not one of the standard general-purpose thematic roles (Agent, Theme, Recipient, Instrument, Experiencer). In a general semantic-role framework it would be recast as a Theme or Stimulus of the verb “reported”, with “Symptom” retained as an additional clinical-ontology tag layered on top.
 - Recommended corrected role set: Doctor=Agent, Medicine=Theme, Patient=Recipient, Headache=Theme/Stimulus (with a separate clinical-concept tag “symptom” applied via a medical ontology rather than the core thematic-role slot).
3. Evaluating potential errors that could arise from incorrect parsing.
 - Prepositional-phrase attachment ambiguity: in “Doctor prescribed medicine to patient,” a parser could mis-attach “to patient” to the wrong head, or in more complex clinical sentences attach a PP to the wrong verb entirely, corrupting the Recipient role assignment.
 - Compound noun mis-segmentation: a term like “blood pressure” in “Medicine reduced blood pressure” could be incorrectly split into two separate objects (“blood” and “pressure”) instead of one clinical entity, which would distort both the Theme role and any downstream clinical concept extraction.
 - Passive-voice misreading: clinical notes frequently use passive constructions (“Medicine was administered to the patient by the nurse”); a parser tuned mainly on active SVO sentences can swap Agent and Theme roles if it fails to detect the passive marker.

- Domain-specific telegraphic style: clinical documentation often drops function words, uses abbreviations, and has elliptical sentences (e.g., “Pt c/o headache, given analgesic”), which general-purpose parsers trained on standard English are prone to mis-parse, again cascading into wrong role labels.
- Consequence: any of these errors can lead to unsafe clinical interpretation downstream — for example, swapping Agent and Recipient could misattribute who administered a drug, which matters for accountability and safety-critical documentation.

4. Recommending methods to improve syntax-driven semantic analysis in healthcare applications.

- Use domain-adapted parsers trained/fine-tuned on clinical corpora (e.g., clinical treebanks) rather than general-English parsers, combined with medical Named Entity Recognition (NER) to correctly bound multi-word clinical entities like “blood pressure” before role labeling.
- Standardize role labeling against an established linguistic scheme such as PropBank or FrameNet instead of ad hoc domain labels, so roles are consistent and interoperable across systems, while still layering clinical-ontology tags (e.g., SNOMED-CT, UMLS concept codes) on top for domain meaning.
- Cross-validate extracted roles against clinical ontologies/knowledge bases to catch implausible role assignments automatically (e.g., a medication being assigned the Agent role would be flagged as anomalous).
- Add explicit handling for passive voice, ellipsis, and clinical abbreviations/shorthand as a preprocessing normalization step before parsing.
- Keep a human-in-the-loop review step for any syntax-to-semantics output that feeds directly into clinical decision-making, since even a well-tuned pipeline should not be fully autonomous in a safety-critical domain.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Understanding of Data	Analysis	Applica tion of Conce pts	Conclus ion	Clarity	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____